

CONTINUUM: Frontend Integration Contract & Specification

PRISM Generative AI Hackathon (3rd Edition 2026-27) | Theme 05: Interruptible Real-Time Agents

Target Role: Frontend Engineer (React + Tailwind)	WebSocket URI: ws://localhost:8000/ws/{session_id}	REST Base: http://localhost:8000/api/v1
Evaluation Scope: Split-Screen HUD, DAG Visualizer (30%)	Session ID: Frontend generated (sess_<id>)	Instant ACK Budget: < 10ms

1. Session Lifecycle, Ownership & Inbound Protocol

- Session ID Ownership:** Frontend generates session_id using format sess_<alphanumeric> (e.g. `sess\_\${crypto.randomUUID().slice(0,8)}` or sess_demo).
- Connection & Handshake:** Frontend opens ws://localhost:8000/ws/{session_id}. Backend immediately responds with SESSION_INIT containing the current DAG.
- Reconnection Protocol:** If connection drops, frontend attempts reconnect with backoff (1s, 2s, 4s). On reconnect to the same session_id, backend re-sends SESSION_INIT to restore state.
- User Inbound Utterance / Interruption:** Send via WebSocket (preferred) or REST POST /api/v1/session/interrupt.

WebSocket Inbound Message:	REST Inbound Request & Instant Response (<10ms):
<pre>// Client -> WS { "action": "utterance", "session_id": "sess_89f2c1", "text": "Actually, make it Bangalore, keep morning" }</pre>	<pre>// POST /api/v1/session/interrupt { "session_id": "sess_89f2c1", "utterance": "Actually, make it Bangalore, keep morning" } // Immediate Response (<10ms) { "event_id": "evt_9a41", "version": 2, "ack_emitted": true, "latency_ms": 3.6 }</pre>

2. Unified WebSocket Event Protocol & Envelope

Every event streamed from the backend adheres strictly to the following frozen envelope structure:

<pre>type CONTINUUMEventType = 'SESSION_INIT' 'ACK' 'INTENT_UPDATE' 'VERSION_UPDATE' 'NODE_UPDATE' 'METRICS_UPDATE' 'CLARIFICATION_REQUEST' 'FINAL_RESPONSE' 'ERROR'; interface CONTINUUMWebSocketEvent { event_type: CONTINUUMEventType; session_id: string; // e.g. 'sess_89f2c1' event_id: string; // e.g. 'evt_9a41c2' version: number; // Monotonic version counter (\$V_1, V_2, V_3\$) timestamp: number; // Unix epoch in milliseconds step_id?: string; // 'p_1', 'p_2', 'sh_1' (where applicable) branch_id?: string; // 'PRIMARY' 'SHADOW_1' 'SHADOW_2' status?: string; // Lifecycle status payload: Record<string, any>; // Complete event-specific schema }</pre>

3. Exhaustive WebSocket Event Payloads (All Event Types Defined)

Event Type	Trigger & Timing	Exact JSON Payload Structure
SESSION_INIT	On WebSocket Connect (0ms)	<pre>{ "event_type": "SESSION_INIT", "payload": { "current_version": 0, "active_tasks_count": 0, "dag": { "session_id": "sess_89f", "nodes": {} } }</pre>
ACK	Instant Fast Path (<10ms)	<pre>{ "event_type": "ACK", "version": 2, "payload": { "text": "Got it, updating...", "latency_ms": 3.6 } }</pre>
INTENT_UPDATE	5-Way Arbiter + IVS (<250ms)	<pre>{ "event_type": "INTENT_UPDATE", "payload": { "delta_type": "MODIFY", "confidence": 0.94, "ivs_score": 0.35, "authorization": "IMPLIED", "affected_fields": ["destination"], "preserved_constraints": ["departure_time: morning"] } }</pre>
VERSION_UPDATE	On Interrupt Arrival (<5ms)	<pre>{ "event_type": "VERSION_UPDATE", "payload": { "previous_version": 1, "current_version": 2 } }</pre>
NODE_UPDATE	Lifecycle State Change (Full Node State)	<pre>{ "event_type": "NODE_UPDATE", "step_id": "p_1", "status": "RUNNING", "payload": { "step_id": "p_1", "tool": "search_flights", "risk": "FREE", "params": { "to": "Bangalore" }, "depends_on": [], "status": "RUNNING", "base_version": 2, "is_shadow": false, "branch_id": "PRIMARY", "output": null, "duration_ms": null } }</pre>
CLARIFICATION_REQ	When IVS > 0.60 / Conf < 0.70	<pre>{ "event_type": "CLARIFICATION_REQUEST", "payload": { "prompt": "Unsure about destination. Search Bangalore or hold?", "suggested_actions": ["Search Bangalore", "Hold"] } }</pre>
METRICS_UPDATE	HUD real-time telemetry	<pre>{ "event_type": "METRICS_UPDATE", "payload": { "pivot_latency_ms": 3.8, "median_pivot_latency_ms": 4.1, "p95_pivot_latency_ms": 7.8, "ivs_score": 0.30, "confidence": 0.94, "token_savings_pct": 72.0, "speculation_hits": 4, "speculation_wasted": 1, "branch_cleanup_time_ms": 0.0, "unnecessary_clarification_rate_pct": 0.0, "regretted_actions": 0, "active_tasks_count": 1 } }</pre>
FINAL_RESPONSE	Workflow Completion	<pre>{ "event_type": "FINAL_RESPONSE", "status": "SUCCESS", "payload": { "summary": "Found morning flight to Bangalore and held seat.", "booking_ref": null, "hold_id": "HLD_9a8", "amount_paid": 0, "completed_steps": ["p_1", "p_2"], "status": "SUCCESS" } }</pre>
ERROR	Policy Blocked / Execution Failure	<pre>{ "event_type": "ERROR", "status": "ERROR", "payload": { "error_code": "POLICY_BLOCKED", "message": "Autonomous booking blocked: IRREVERSIBLE requires EXPLICIT auth & IVS < 0.60.", "step_id": "p_3", "recoverable": true, "suggested_action": "Confirm explicitly" } }</pre>

4. DAG Data Model & Node Lifecycle UI Specification (All 8 States)

Every NODE_UPDATE contains the complete node state matching DAGNodeModel so frontend never needs to guess missing properties:

```
interface DAGNodeModel {
  step_id: string;           // Unique node ID ('p_1', 'p_2', 'sh_1')
  tool: string;              // 'search_flights' | 'hold_seat' | 'confirm_booking' | 'search_cabs' | 'search_hotels'
  params: Record<string, any>; // e.g. { to: 'Bangalore', slot: 'morning' }
  risk: 'FREE' | 'STAGEABLE' | 'MUTATING' | 'IRREVERSIBLE';
  depends_on: string[];      // Parent dependency step_ids, e.g. ['p_1']
  status: 'CREATED' | 'RUNNING' | 'COMPLETED' | 'INVALIDATED' | 'CANCELLED' | 'SHADOW' | 'PROMOTED' | 'ABANDONED';
  base_version: number;      // Originating version
  is_shadow: boolean;        // Speculative branch flag
  branch_id: string;         // 'PRIMARY' | 'SHADOW_1'
  output?: any;              // Result payload when COMPLETED
  error?: string;            // Cancellation/Invalidation reason
  duration_ms?: number;      // Execution duration
}
```

Node Status	Tailwind / CSS Styling Guideline	Lifecycle Meaning & UI Behavior
CREATED	border-slate-600 bg-slate-900/40 text-slate-300	Queued in DAG, waiting for parent dependencies
RUNNING	border-blue-500 bg-blue-950/20 text-blue-300 animate-pulse	Task active in asyncio task registry
COMPLETED	border-emerald-500 bg-emerald-950/20 text-emerald-300	Task successfully executed & verified
INVALIDATED	border-rose-500 line-through opacity-60 text-rose-300	Surgically pruned by DAG dependency tracker on parameter change
CANCELLED	border-rose-500 line-through bg-rose-950/20 text-rose-300	Aborted immediately via task.cancel() on retraction
SHADOW	border-purple-500 border-dashed bg-purple-950/20 text-purple-300	Read-only speculative background pre-fetch
PROMOTED	border-amber-400 bg-amber-950/20 text-amber-300 ring-1 ring-amber-400	Speculative shadow branch adopted into active primary plan
ABANDONED	border-slate-700 opacity-40 text-slate-500	Stale Gate discarded result (\$V_{task} != V_{curr}\$)

5. Split-Screen Comparison: Baseline vs CONTINUUM

Component	Left Panel: Vanilla Baseline Agent	Right Panel: CONTINUUM Engine
Interrupt Handling	Discards all progress; full sequential re-prompt	Instant ACK (<10ms), surgical DAG reuse
Tokens Consumed	Accumulates 1,250 tokens per turn	Uses 350 tokens per turn (72% Savings)
Pivot Latency	High latency (~420ms LLM roundtrip)	Sub-millisecond local dispatch (~3.6ms)
Safety Gating	No Policy Engine; risks committing old actions	Stale Gate + Two-Phase Effect Ledger (0 Regrets)

6. Mock WebSocket Event Sequence for Frontend Development

Frontend developers can use this standalone sequence to test the complete live UI without running the backend:

```
// 1. Initial Prompt: 'Find flights to Delhi tomorrow morning'
WS.emit({ event_type: 'ACK', version: 1, payload: { text: 'Got it, updating...', latency_ms: 3.4 } });
WS.emit({ event_type: 'INTENT_UPDATE', version: 1, payload: { delta_type: 'NEW_GOAL', ivs_score: 0.15 } });
WS.emit({ event_type: 'NODE_UPDATE', step_id: 'p_1', status: 'RUNNING', payload: { step_id: 'p_1', tool: 'search_flights', risk: 'FREE', params: { to: 'Delhi', slot: 'morning' }, depends_on: [], status: 'RUNNING', base_version: 1, is_shadow: false, branch_id: 'PRIMARY' } });
WS.emit({ event_type: 'NODE_UPDATE', step_id: 'sh_1', status: 'SHADOW', payload: { step_id: 'sh_1', tool: 'search_cabs', risk: 'FREE', hypothesis: 'Delhi airport transit', params: { to: 'Delhi Airport' }, depends_on: [], status: 'SHADOW', base_version: 1, is_shadow: true, branch_id: 'SHADOW_1' } });

// 2. User Interrupt at t=1200ms: 'Actually, make it Bangalore, keep morning'
WS.emit({ event_type: 'ACK', version: 2, payload: { text: 'Got it, updating...', latency_ms: 3.8 } });
WS.emit({ event_type: 'NODE_UPDATE', step_id: 'p_1', status: 'CANCELLED', payload: { step_id: 'p_1', tool: 'search_flights', risk: 'FREE', params: { to: 'Delhi' }, depends_on: [], status: 'CANCELLED', base_version: 2, is_shadow: false, branch_id: 'PRIMARY', error: 'Surgically invalidated by pivot' } });
WS.emit({ event_type: 'NODE_UPDATE', step_id: 'sh_1', status: 'ABANDONED', payload: { step_id: 'sh_1', tool: 'search_cabs', risk: 'FREE', params: { to: 'Delhi Airport' }, depends_on: [], status: 'ABANDONED', base_version: 1, is_shadow: true, branch_id: 'SHADOW_1', error: 'Stale Gate: Speculative Delhi cab discarded' } });
WS.emit({ event_type: 'NODE_UPDATE', step_id: 'p_1_blr', status: 'RUNNING', payload: { step_id: 'p_1_blr', tool: 'search_flights', risk: 'FREE', params: { to: 'Bangalore', slot: 'morning' }, depends_on: [], status: 'RUNNING', base_version: 2, is_shadow: false, branch_id: 'PRIMARY' } });
WS.emit({ event_type: 'NODE_UPDATE', step_id: 'p_1_blr', status: 'COMPLETED', payload: { step_id: 'p_1_blr', tool: 'search_flights', risk: 'FREE', params: { to: 'Bangalore', slot: 'morning' }, depends_on: [], status: 'COMPLETED', base_version: 2, is_shadow: false, branch_id: 'PRIMARY', output: { flight_id: 'FL_BLR_702', price: 5400 } } });
WS.emit({ event_type: 'METRICS_UPDATE', version: 2, payload: { pivot_latency_ms: 3.8, median_pivot_latency_ms: 4.1, p95_pivot_latency_ms: 7.8, ivs_score: 0.30, confidence: 0.94, token_savings_pct: 72.0, speculation_hits: 4, speculation_wasted: 1, branch_cleanup_time_ms: 0.0, unnecessary_clarification_rate_pct: 0.0, regretted_actions: 0, active_tasks_count: 1 } });
```